МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

**Федерального государственного автономного образовательного учреждения
высшего образования**

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ МИСИС»

Институт компьютерных наук

Кафедра Инженерной кибернетики

Лабораторная работа №1

по дисциплине «Современные интеллектуальные сетевые сервисы»

Проектирование AI-сервиса на FastAPI

Выполнил:
Студент группы МПИ-25-1-1
Сульповар Михаил Леонидович

Проверил:
Преподаватель

_____

Оценка: _____

Москва, 2026

Содержание

1 Цель работы **2**

2 Краткое описание архитектуры API **2**
2.1 Программные компоненты 2
2.2 Компонентная схема 3
2.3 Входные и выходные данные 3
2.4 Контур инференса и небольшое изменение логики 3
2.5 Хранение, наблюдаемость и ограничения 4

3 Исходный код приложения **5**

4 Реализованные endpoints **5**
4.1 Чем /health отличается от /model-info? 6
4.2 Почему некорректный запрос не должен доходить до модели? . 6
4.3 Выбор между 200 и 201 6

5 Swagger UI **6**

6 Успешный запрос /predict **7**

7 Ошибка валидации Pydantic **8**

8 Ошибка 404 Not Found **9**

9 Таблица тест-кейсов **10**

10 Таблица API-контракта **10**

11 Вывод **11**

1 Цель работы

Научиться проектировать программную архитектуру AI-сервиса, определить место API в общей интеллектуальной системе и реализовать REST API на FastAPI. Освоить передачу JSON, входные и выходные схемы Pydantic, структурную и бизнес-валидацию, HTTP-коды, хранение результатов и описание контракта OpenAPI.

2 Краткое описание архитектуры API

Разработан сервис Agro Scoring API для оценки риска сельскохозяйственного предприятия. Потребители: CRM банка, веб-портал, мобильное приложение, внутренние банковские сервисы и BI-система. Обмен выполняется по HTTP в формате JSON; для внешнего развертывания требуется HTTPS.

В работе используется учебная модель на правилах, а не обученная ML-модель. Прогноз возвращается синхронно в рамках одного запроса. Длительные операции (обучение модели, пакетный инференс) в прототип не включены: в дальнейшем им потребуется очередь задач и отдельный обработчик. Асинхронная функция middleware обслуживает HTTP-запросы, но сама по себе не делает инференс фоновой задачей.

2.1 Программные компоненты

Ответственности разделены по директориям без микросервисов и дополнительных фреймворков.

Таблица 1: Компоненты приложения

Файл	**Ответственность**
`app/main.py`	Сборка FastAPI, подключение маршрутов, логирование, middleware времени обработки.
`app/api/routes.py`	HTTP-методы, path/query-параметры, статусы и преобразование исключений сервиса.
`app/schemas/prediction.py`	Входные и выходные модели Pydantic, ограничения полей.
`app/models/risk.py`	Расчет риска и метаданные учебной модели.
`app/services/scoring.py`	Бизнес-валидация, инференс, постпроцессинг и сохранение результата.
`app/repositories/predictions.py`	Хранилище прогнозов и выборка последних результатов.

2.2 Компонентная схема

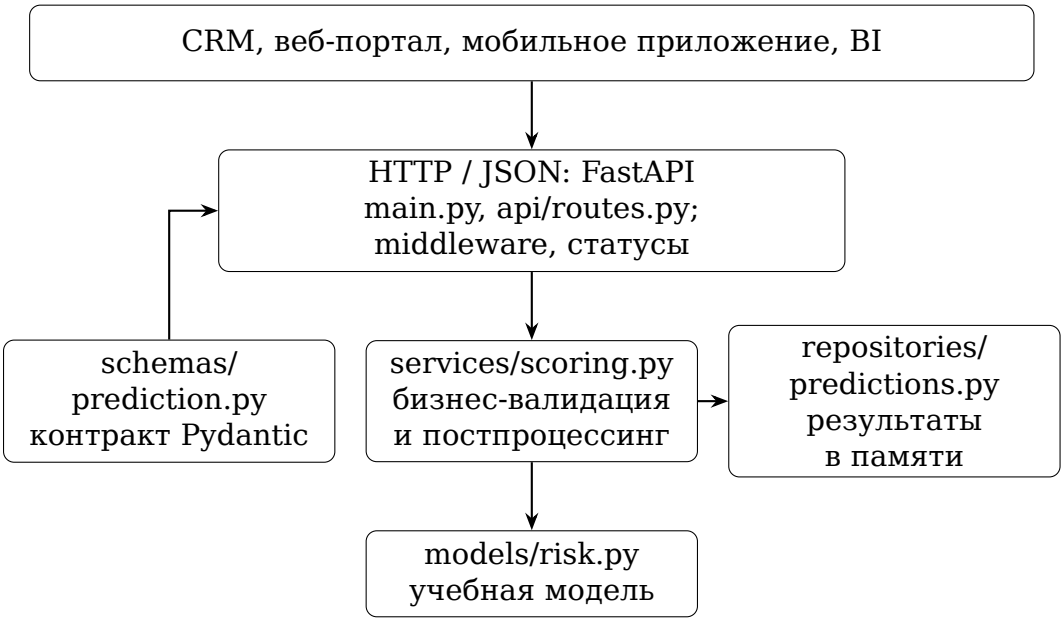


Рис. 1: Компонентная схема сервиса агроскоринга

2.3 Входные и выходные данные

Все девять полей запроса обязательны. Строки очищаются от пробелов по краям; пустые значения не допускаются. Числовые значения должны быть конечными.

Таблица 2: Ограничения входных данных

Поле	Тип	Ограничение
`farm_id`, `region`, `crop_type`	string	Не менее одного символа после удаления внешних пробелов.
`area_ha`	number	Больше нуля.
`temperature_avg`	number	От -60 до 60 включительно.
`precipitation_mm`, `debt`	number	Не меньше нуля.
`payment_delay_days`, `previous_defaults`	integer	Не меньше нуля.

Регион дополнительно проверяется по справочнику `Krasnodar`, `Rostov`, `Stavropol`. Ответ включает `request_id`, `farm_id`, `risk_score`, `risk_level`, `recommendation`, `model_version`.

2.4 Контур инференса и небольшое изменение логики

Последовательность обработки запроса:

1. Pydantic проверяет тело запроса до вызова обработчика; нарушение контракта дает 422.

2. Сервис проверяет готовность модели; если она недоступна, возвращается 503.
3. Регион проверяется по бизнес-справочнику; неизвестный регион дает 400.
4. Создается UUID4 и записывается начало обработки в журнал.
5. Вычисляется оценка риска, категория и рекомендация.
6. Прогноз сохраняется в хранилище; результат обработки записывается в журнал.
7. Возвращается 201 и заголовок `Location` с путем созданного прогноза.

Исходная логика сохранена. Единственное дополнение к расчету: средняя температура ниже $5\,^{\circ}$C или выше $35\,^{\circ}$C увеличивает оценку на $0{,}05$. Это условное учебное правило, его эффективность на реальных данных не проверялась. Версия модели изменена на 1.1.

Таблица 3: Правила расчета риска

Условие	Вклад
Базовая оценка	0,10
Просрочка больше 30 дней	+0,30
Есть хотя бы один предыдущий дефолт	+0,30
Задолженность больше 5 000 000	+0,20
Осадки меньше 100 мм	+0,10
Средняя температура ниже 5 или выше 35 градусов	+0,05

Сумма ограничивается сверху единицей и округляется до двух знаков. Категории: `low` при $s < 0{,}3$, `medium` при $0{,}3 \le s < 0{,}7$, `high` при $s \ge 0{,}7$. Рекомендации: стандартное рассмотрение, дополнительная проверка, ручное рассмотрение соответственно.

2.5 Хранение, наблюдаемость и ограничения

Результаты хранятся в словаре одного экземпляра приложения. Блокировка защищает запись и создание снимка словаря при параллельных запросах. Список возвращается в обратном порядке сохранения: сначала новые результаты. Фильтрация выполняется до ограничения количества.

Middleware добавляет `X-Process-Time` в секундах и записывает метод, путь, статус и длительность. Журнал успешного инференса содержит идентификаторы запроса и хозяйства, оценку и категорию риска. Полное тело запроса в журнал не записывается.

При перезапуске данные теряются; несколько процессов имеют независимые хранилища. Память растет с числом прогнозов. Для дальнейшего развития нужны БД, политика удаления старых записей, аутентификация, контроль доступа и проверка качества настоящей модели. В учебном запуске используется один процесс. Обученную модель можно подключить в модуле модели; подключение БД потребует адаптации хранилища и сборки приложения при сохранении HTTP-контракта.

3 Исходный код приложения

Ссылка на проект: добавляется после загрузки проекта в собственный репозиторий. Перед сдачей необходимо заполнить команду `\RepositoryURL` в начале этого файла. Код поставляется в архиве `agro_scoring.zip`. Директории отражают отдельные ответственности, импорты идут от корневого пакета `app`. Нет циклических импортов между API и сервисом.

```
agro_scoring/
    app/
        main.py
        api/routes.py
        schemas/prediction.py
        models/risk.py
        services/scoring.py
        repositories/predictions.py
    tests/test_api.py
    requirements.txt
    requirements-dev.txt
    pyproject.toml
    README.md
    report/
        main.tex
        images/
```

Листинг 1: Структура проекта

В директориях Python также находятся `__init__.py`. Проверка стиля: Ruff, длина строки 88, правила E/W/F/I/UP/B/SIM. Докстринги написаны в свободной форме. Для запуска нужен Python 3.12 или новее.

```
python -m pip install -r requirements-dev.txt
python -m uvicorn app.main:app --reload
python -m pytest -q
python -m ruff check .
python -m ruff format --check .
```

Листинг 2: Установка, запуск и проверка из корня проекта

Для обычного запуска достаточно `requirements.txt`; зависимости тестов и Ruff находятся в `requirements-dev.txt`. LaTeX собирается XeLaTeX дважды из директории `report`; используются шрифты DejaVu и изображения из `images`.

4 Реализованные endpoints

Таблица 4: Маршруты API

Метод	Endpoint	Назначение
GET	`/health`	Доступность процесса API.
GET	`/model-info`	Метаданные и состояние модели.
POST	`/predict`	Создание и сохранение прогноза.
GET	`/predictions/{request_id}`	Чтение результата по ID.
GET	`/predictions`	Последние прогнозы с фильтрацией.

4.1 Чем /health отличается от /model-info?

`/health` возвращает `{"status": "ok"}` и проверяет доступность API. `/model-info` описывает модель и возвращает состояние `ready` или `unavailable`. API может отвечать на запросы, когда модель недоступна; тогда корректный запрос `/predict` получает 503.

4.2 Почему некорректный запрос не должен доходить до модели?

Отрицательная площадь физически невозможна. Такие признаки нарушают ожидаемый формат данных, могут вызвать ошибку вычислений или недостоверный результат. Валидация предотвращает бесполезный инференс и сохранение ошибочного прогноза; клиент получает указание на конкретное поле. Бизнес-валидатор отдельно отклоняет формально корректный, но неподдерживаемый регион.

4.3 Выбор между 200 и 201

Выбран 201 Created: каждый успешный запрос создает новый сохраненный прогноз с UUID. Заголовок `Location` указывает его адрес. Все операции чтения возвращают 200. Для инференса без создания сохраняемого ресурса можно было бы выбрать 200, но в этой реализации семантика создания используется последовательно.

5 Swagger UI

Документация доступна по адресу `http://127.0.0.1:8000/docs`, спецификация по `http://127.0.0.1:8000/openapi.json`. Swagger показывает маршруты, схемы, ограничения и ответы. Значения фильтра категории описаны типом `Literal`; некорректная категория дает 422.

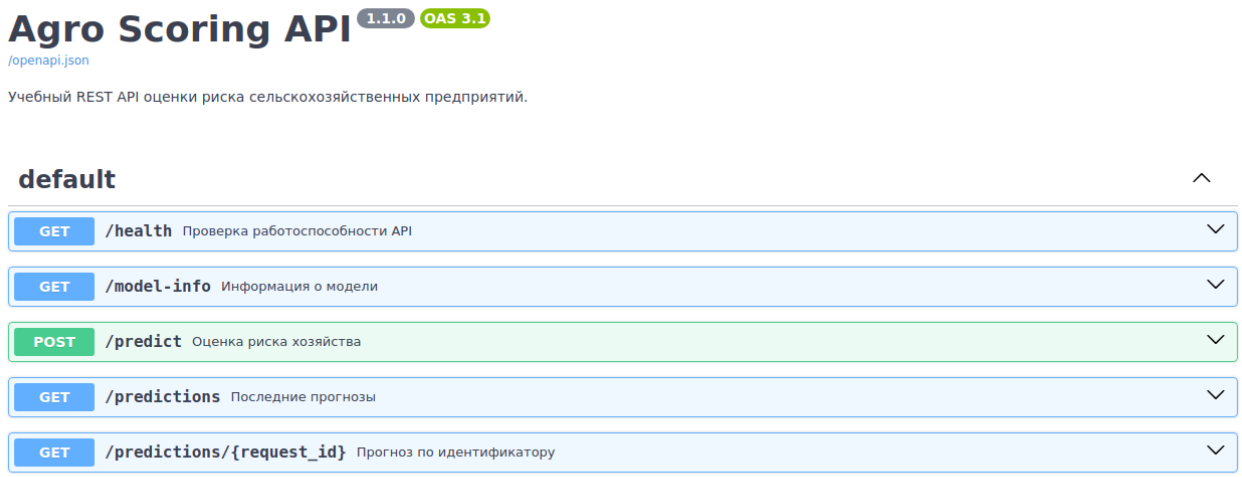


Рис. 2: Swagger UI работающего приложения

6 Успешный запрос /predict

```
{
  "farm_id": "FARM-101",
  "region": "Rostov",
  "crop_type": "sunflower",
  "area_ha": 1200,
  "temperature_avg": 38,
  "precipitation_mm": 80,
  "payment_delay_days": 45,
  "previous_defaults": 1,
  "debt": 6500000
}
```

Листинг 3: Тело запроса

Для выбранных данных сумма составляет $0{,}10+0{,}30+0{,}30+0{,}20+0{,}10+0{,}05 = 1{,}05$. После ограничения получено 1,00, категория `high`, рекомендация ручного рассмотрения.



Рис. 3: Успешное выполнение POST /predict в Swagger UI, HTTP 201

UUID создается при каждом запросе; он не фиксирован в программе.

7 Ошибка валидации Pydantic

В исходном запросе площадь заменена на -100. Получен 422, модель не вызывается и результат не сохраняется. Массив `detail` содержит тип нарушения, положение поля, сообщение, исходное значение и ограничение.

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": [
        "body",
        "area_ha"
      ],
      "msg": "Input should be greater than 0",
      "input": -100,
      "ctx": {
        "gt": 0.0
      }
    }
  ]
}
```

Листинг 4: Ответ Pydantic на отрицательную площадь

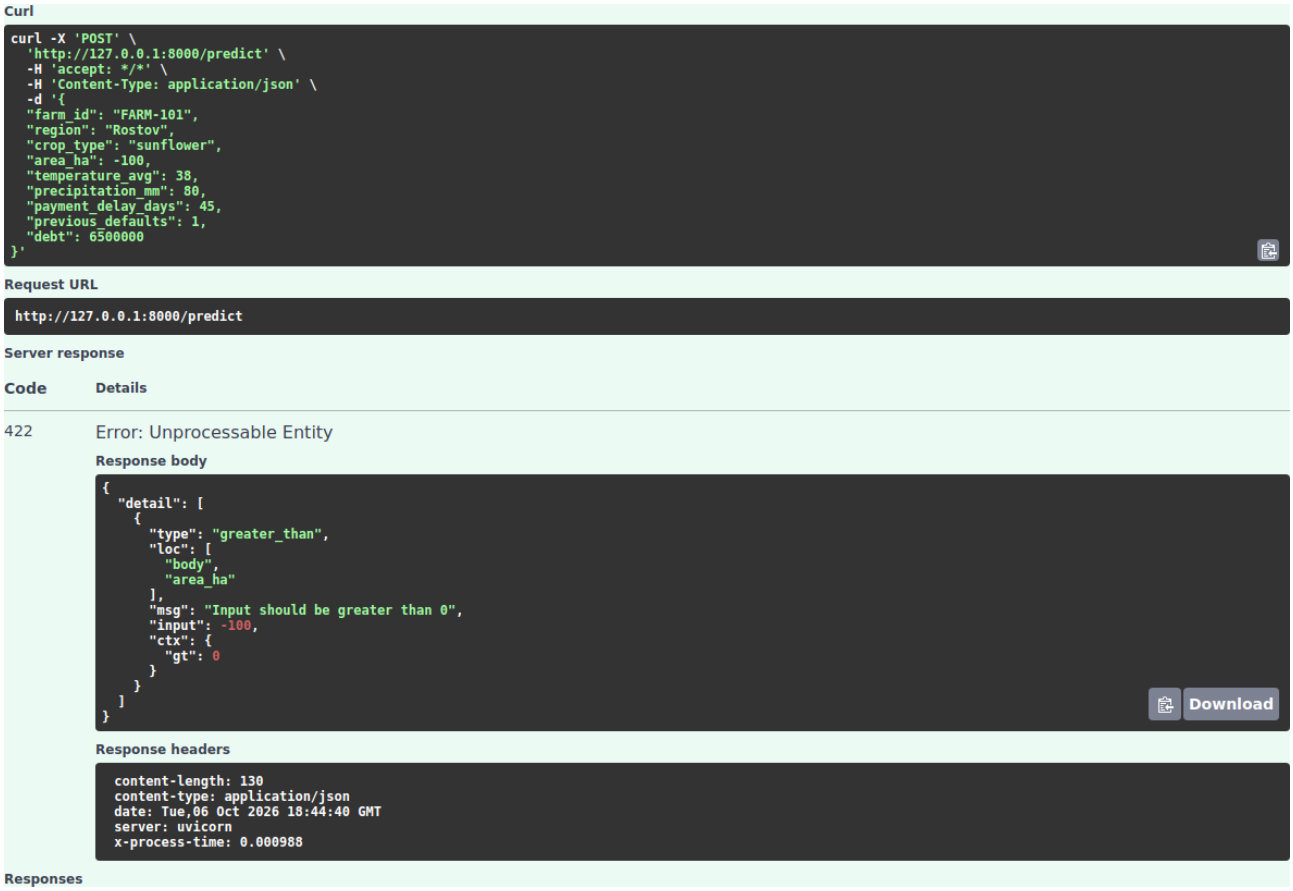


Рис. 4: Ошибка структурной валидации в Swagger UI, HTTP 422

8 Ошибка 404 Not Found

Выполнен запрос к `/predictions/unknown-id`. Такого идентификатора в хранилище нет; ответ содержит `detail: Prediction not found`.

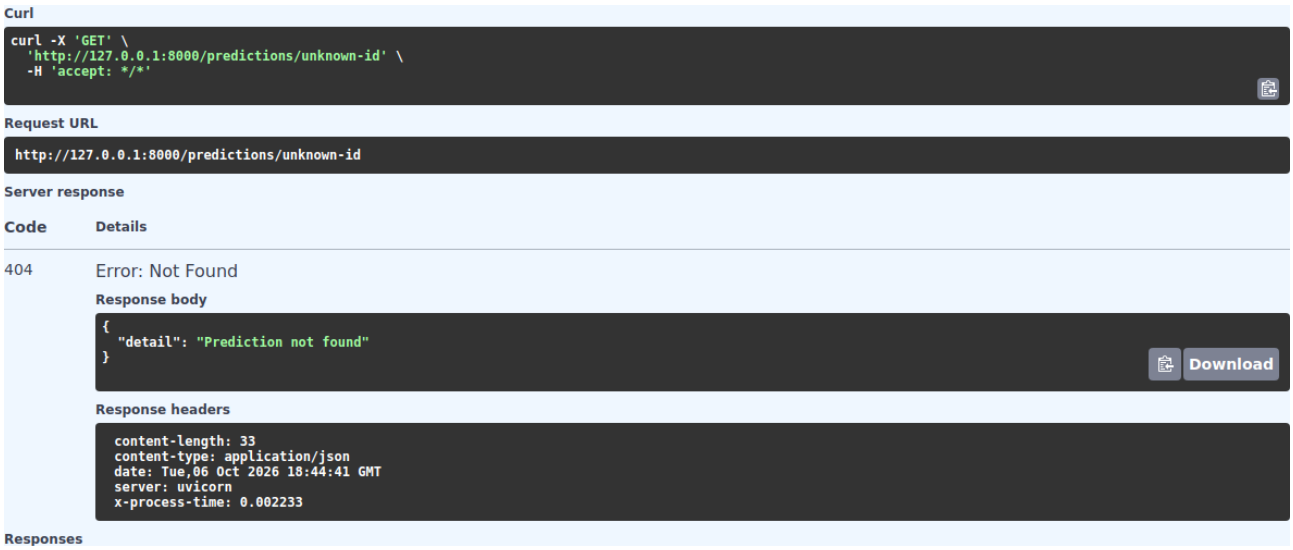


Рис. 5: Обращение к отсутствующему прогнозу в Swagger UI, HTTP 404

9 Таблица тест-кейсов

Проверки выполнены с помощью FastAPI TestClient. Все строки таблицы получены при фактическом выполнении запросов. Дополнительно набор pytest проверяет порядок результатов, фильтрацию, сохранение, заголовки, границы категорий, температурное правило и OpenAPI: 22 проверки пройдены.

Таблица 5: Результаты HTTP-проверок

№	Запрос / условие	Ожидалось	Получено	Итог
1	GET /health	200	200	OK
2	GET /model-info	200	200	OK
3	Корректный POST /predict	201	201	OK
4	area_ha = -100	422	422	OK
5	Неизвестный регион	400	400	OK
6	Существующий request_id	200	200	OK
7	Неизвестный request_id	404	404	OK
8	limit = 2	200	200	OK
9	risk_level = high	200	200	OK
10	limit = -5	422	422	OK
11	limit = 101	422	422	OK
12	risk_level = extreme	422	422	OK
13	risk_level = medium, limit = 5	200	200	OK
14	Отсутствует поле debt	422	422	OK
15	Модель недоступна	503	503	OK

Ruff check и Ruff format –check выполнены успешно. При тестировании использовались версии зависимостей, зафиксированные в файлах требований.

10 Таблица API-контракта

Таблица 6: Контракт API

Endpoint	Метод	Вход	Выход	Успех	Ошибки
`/health`	GET	Нет	HealthResponse	200	500
`/model-info`	GET	Нет	ModelInfoResponse	200	500
`/predict`	POST	JSON: FarmRequest	PredictionResponse	201	400, 422, 503, 500
`/predictions/{request_id}`	GET	path: request_id	PredictionResponse	200	404, 422, 500
`/predictions`	GET	query: limit, risk_level	Массив PredictionResponse	200	422, 500

Параметр `limit`: целое число от 1 до 100, по умолчанию 10. `risk_level`: необязательный фильтр `low`, `medium`, `high`. Path-параметр `request_id` имеет тип строки; произвольный отсутствующий идентификатор возвращает 404. Код 422 для path-маршрута присутствует в автоматически создаваемой спецификации FastAPI; отдельное UUID-ограничение не введено.

Бизнес-ошибки 400, 404 и 503 явно описаны через `responses` и схему ErrorResponse. Ошибки валидации 422 добавляются FastAPI автоматически.

Код 500 указан как возможный непредвиденный серверный сбой, а не как штатный бизнес-ответ; его отдельная схема в OpenAPI не объявлялась.

11 Вывод

Разработан и проверен учебный AI-сервис агроскоринга. Сборка приложения, HTTP-маршруты, контракт данных, модель, бизнес-логика и хранилище разделены по самостоятельным модулям. Структурная и бизнес-валидация выполняются до инференса; прогноз сохраняется под UUID и доступен повторно. Реализованы пять обязательных endpoints, корректные HTTP-коды, фильтрация, выдача последних результатов, логирование и измерение времени обработки.

REST API служит общей точкой входа для потребителей, фиксирует контракт и скрывает внутреннюю реализацию модели. Благодаря разделению ответственностей модель и способ хранения можно развивать при сохранении клиентского интерфейса. Прототип остается ограниченным учебным решением: модель является набором правил, результаты хранятся только в памяти одного процесса.